Lab 05

Classes and Objects – A Deeper Look

Objectives:

At the end of this lab the student should be able to:

- Use the Math class and its static members.
- Declare and use static methods.
- Work with setters, getters, overloaded constructors, and static methods.
- Generate documentation using the Javadoc tool.

==========

Exercise 1: Geometric Shapes: the Circle

Create a class called Circle. The class has an attribute radius, which default to 1 by the first *no argument constructor*. The class also has another *constructor* that instantiates the radius to a given value. Provide *set* and *get* methods. The *set* method should verify that radius is greater than or equal to 0.0 and less than 20.0, otherwise it sets radius to 1. Provide methods that calculate the circumference $2\pi r$, and the area πr^2, of the circle (use Math.*PI* and Math.*pow*). Write another class called *TestCircle* that tests the Circle Class.

Exercise 2: Constructor Overloading

In a *Student Admission System,* an applicant chooses from among many offered majors such as computer science, computer engineering, physics, math, industrial engineering … etc.

The Major class is represented by the following attributes: majorId (int), title (String), hostDeptId(int), totalCreditHours (int).

a) Write Java code for the Major class, including the setters, getters and the fully parameterized constructor.
b) Add a no-argument constructor that sets majorId to 999, the title to "Unknown," the hostDeptId to 888 and the totalCreditHours to 0.
c) Write a method called display() that outputs the values of a Major instance, each on a separate line and which is properly labelled.
   Title: Computer Science
   Credit Hours: 120
d) Write a class called MajorTest that creates some Major instances and displays their contents using the above method. You should utilize the two overloaded constructors.

Exercise 3: Using static methods

1. Trace the following method to figure out its task**.**

```
public int digits(int x) {
            double n=0;
            while (x!=0){
                  x=x/10;
                  n++;
            }
            return n;
 }
```

The method above returns _____.

2. Does the above method have a compilation error? _____
3. Create a new project *project43*, and create new class *IntNumbers*. After fixing the error in method *digits*, copy that method to your class.
4. Your main method in this class may look as follows

```
public static void main(String arg[]){
            Scanner keyboard=new Scanner(System.in);
            System.out.print("Enter a value for n: ");
            int n= keyboard.nextInt();
            System.out.println(digits(n));
            System.out.println(Math.sqrt(n));
            keyboard.close();
}
```

5. Do you have any compilation error as a result of calling digits method? Yes [ ] No[ ]
   If yes , what is the problem ?_____
6. Can you explain why didn't the program have an error for calling the method `Math.sqrt`, but it did have a compilation error when calling the method `digits`?
   (Tip: Explore the `Math` class in the Java API and find/explore its method `sqrt`)
   _____
   - Add the word "`static` " after the word "`public`" and before "`int`" in method `digits`.
7. It is concluded that

static methods (like main method) only call _____

8. Does it matter if you put *digits* method before or after the main method? Yes[ ] No[ ]
9. How can we define the variable *n* twice in our program (in the main and in digits method)?
   _____
   _____
10. Create a new class called *IntNumbersTest,* move the main method from *IntNumbers* to this new class.(now class *IntNumbers* only includes *digits* method)
    Are we still able to call method *digits* now ? Yes[ ] No[ ]
11. If the method *digits* is still static in class *IntNumbers*, which of the following is the recommended way to call the method *digits*
    a) *IntNumbers num=new IntNumbers();*
       System.out.println(num.digits(n));

    b) *System.out.println(IntNumbers.digits(n));*

    The correct answer _____

    What is the warning message reported by Eclipse when you used this methodology to access digits()? _____.
12. It is concluded that

There are three ways to call a method:
1)_____
2)_____
3)_____

Exercise 4: Work with class methods - simple GUI input/output

Simple dialogs appear in popup windows. Dialogs include short messages or information screens, confirmation boxes, and input prompts for string information. Java's ***Swing*** library uses the **JOptionPane** class to provide static methods for each type of dialog. Each method has a first parameter that points to a parent (i.e. *window that it appears in*) or null (*default to the current window*).

- ***showMessageDialog()*** has a second parameter for the message to be displayed.

```
JOptionPane.showMessageDialog(null,
"This is just a message" );
```



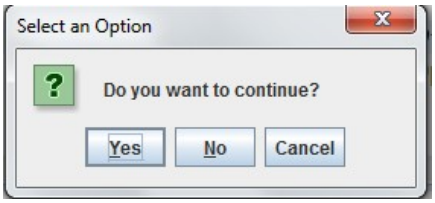
- ***showInputDialog()*** has a second parameter for the message to be displayed. Optionally two more parameters can be used to set a dialog title and to select the dialog's icon. There is both an 'OK' button and a 'Cancel' button for dialog completion. Any information typed into the entry box is returned as a string.

```
String name=JOptionPane.showInputDialog(null,
"What's your name");
```



- ***showConfirmDialog()*** has a second parameter for the message to be displayed. By default the dialog displays three buttons: Yes, No and Cancel. Optionally two more parameters can be used to set a dialog title and to select the dialog's icon.

```
int response=JOptionPane.showConfirmDialog(null,
"Do you want to continue?");
```



The JOptionPane.YES_NO_OPTION constant can be used to restrict the displayed buttons to Yes and No only.

```
int response=JOptionPane.showConfirmDialog(null,
     "Do you want to continue?","RandomApp",
     JOptionPane.YES_NO_OPTION);
```



The returned value of type int can be compared to the static constants: YES_OPTION, NO_OPTION, CANCEL_OPTION, OK_OPTION, …. etc.

4.1) Create a Java project called **project4,** create a class called **Ex4**, and copy the following

```
import javax.swing.JOptionPane;
public class Ex4 {

	public static void main(String[] args) {
		int n1, n2, sum, response;
		String firstNumber, secondNumber;
		do {
			// obtain user input (as String) from JOptionPane input dialogs
			firstNumber = JOptionPane.showInputDialog("Enter first integer");
			secondNumber = JOptionPane.showInputDialog("Enter second integer");

			// convert String inputs to int values for use in a calculation
			n1 = Integer.parseInt(firstNumber);
			n2 = Integer.parseInt(secondNumber);

			sum = n1 + n2;
			JOptionPane.showMessageDialog(null, "Sum=" + sum);

			response = JOptionPane.showConfirmDialog(null,
					"Do you want to continue?");
		} while (response == JOptionPane.YES_OPTION);
	}
}
```

- Run the above application.
- The *showInputDialog* method returns data of type _____.
- To covert a String to integer, we use a method called _____.
- Name the static methods and constants used in the above application? _____.
- Did you use a *class* or an *instance* to access these static methods/constants? _____.
- Modify the *showConfirmDialog*() to display Yes and No buttons only (i.e. do not display the Cancel button).
- Extend the application to obtain the product, in addition to the sum.

4.2) Modify your application such that you define your numbers as `double` instead of `int`. Use the static method: Double.parseDouble() instead of Integer.parseInt().

Exercise 5: Working with SecureRandom

In this exercise, you will build an application that helps elementary school students learn multiplication. The application uses simple GUI interfaces, as explained in *Exercise 4* above.

1. Create a new project and call it *project5*, and then create a new class called *RandomApp.*
2. Using class *SecureRandom,* write a method *generateRandom* that returns a random number between 2 and 10.

   Note: Consult the Java API to explore which method of class *SecureRandom()* to use.
3. Write a method called *generateQuestion,* which uses *generateRandom* to generate random numbers, then asks the user with a question such as

   How much is 6 times 7?

   The method checks the user's answer and then returns true if the answer is correct, or false otherwise.

Create a new class called RandomTest, which has a main method in which you create an instance of RandomApp, and then use the *generateQuestion* method to ask the student a question. If the student's answer is correct, display the message "Well done. Correct answer"; otherwise, display "Sorry. Try again,". At the end, ask the user if s/he wants to continue. If the choice is 'Yes', repeat the above, otherwise terminate the loop and stop.

Exercise 6: Working with Javadoc

JavaDoc is a tool that generates *HTML* documentation from Javadoc comments in the code (similar to the reference pages at *docs.oracle.com*; for example the documentation about the SQL package is found at http://docs.oracle.com/javase/8/docs/api/java/sql/package-summary.html). In this lab, we will go over how to write basic Javadoc comments and how to use features of Javadoc to generate *HTML* documentation.

JavaDoc Comments

- Javadoc recognizes special comments /**....*/ which are highlighted blue by default in Eclipse (Regular comments / /and /*...*/ are highlighted green).
- Javadoc allows you to attatach descriptions to classes, constructors, fieilds, interfaces and methods in the generated HTML documentation by placing Javadoc comments directly before their declaration statements.

Javadoc Tags

- ***Tags*** are keywords recognized by Javadoc, which definne the type of information that follows.
- Tags come after the description (separated by a newline).
- Here are some common pre-definned tags:
    - ***@author [author name]***-identifies author(s) of a class or interface.
    - ***@version [version]***-version info of a class or interface.
    - ***@param [argument name][argument description]***-describes an argument of method or constructor.
    - ***@return [description of return]***-describes data returned by method (unnecessary for constructors and void methods).
    - ***@exception [exception thrown][exception description]***-describes exception thrown by method.
    - ***@throws [exception thrown][exception description]***-same as ***@exceptio***n.
- Here's an example using Javadoc comments to describe a class, a field and a constructor:

Javadoc In Eclipse

- Eclipse can generate Javadoc comments for classes and methods.

1. Place the cursor in the text of class or method declaration.
2. Right click → Source →Generate Element Comment.
3. Javadoc comments with appropriate tags are generated, but you still have to write the descriptions.

Creating JavaDoc for 'Lab4 Exercise3 Project'

1. Add the following to class *Person* before the class definiition.

```
/**
* @author Your name and Student ID
* @version 1.0
*
*/
```

2. Now let us add documentation for the parameterized constructor.

3. Put your cursor within the constructor declaration, right-click, select source, then Generate Element Comment. This action will add the following lines just above the constructor:

```
/**
* @param firstName
* @param lastName
* @param age
 */
```

4. Now you may add more details for this automatically generated initial documentation. Do not worry about having the documentation lines too long; you may just use the Format action to split the lengthy line into smaller lines.

   e.g.:

```
/**
* @param firstName
*          The first name of the person
* @param lastName
*          The last name of the person
* @param age
*          The age of the person.
 */
```

5. Now let's add a description for the fields in this class. Have your cursor positioned within the declaration of the firstName field, then select Source Generate Element Comment. Thihs will generate an empty comment block above the firstName fieeld

```
/**
 *
 */
private String firstName;
```

6. Add your description for all the fieilds and format it:

```
/**
 * The first name of the person
 */
private String firstName;
/**
 * The last name of the person
 */
private String lastName;
/**
 * The age of the person.
 */
private int age;
```

7. Add the following before the setAge method: (*T
 tag is used to create a line break)*

```
/**
 * This method sets the age field for a Person object. It ensures that the
* <br>
* argument is positive. If it is negative, the method does nothing.
* @param years
*                  An integer value that represents the age in years.
*/
public void setAge(int years) {
:
:
```

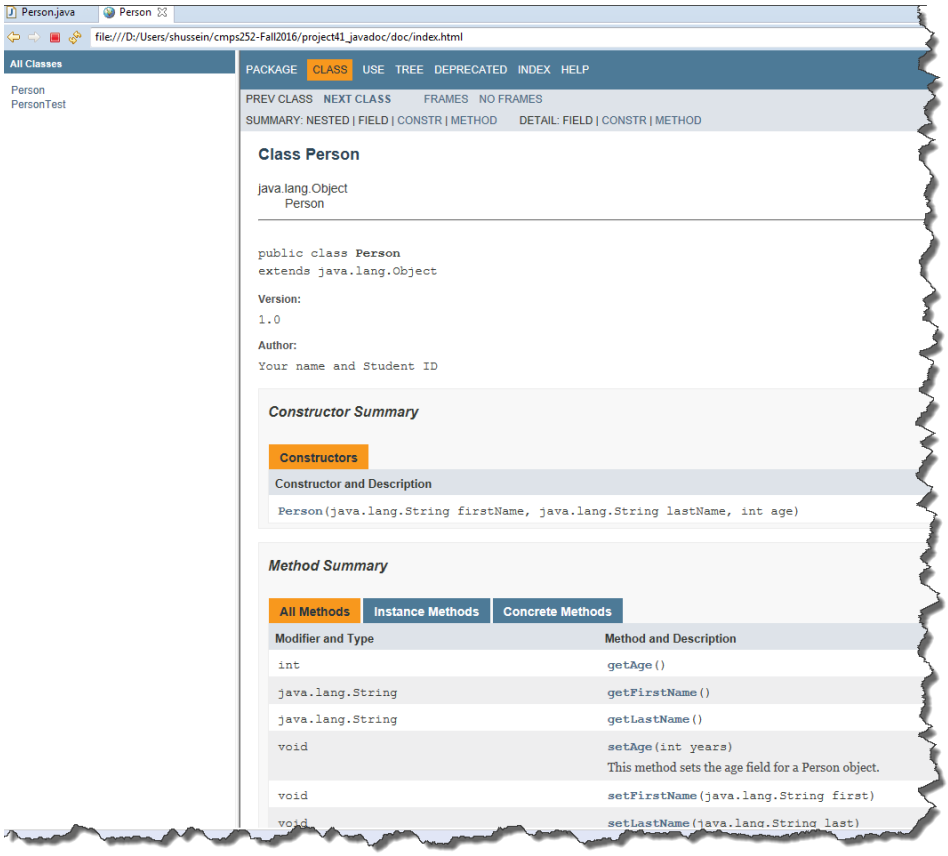
Generating JavaDoc for the above project

Afteter having completed all the documentation, you may generate your HTML Java documentation by doing the following steps:

1. From the package explorer view, select the project for which you want to generate the Javado and from the 'Project' menu, select "*Generate Javadoc...*". A wizard starts and loads a dialog, in the second window, you might need to specify the location of the Javadoc command, usually, it is in the bin folder of the JDK, for example:

   (*d:\Program Files\Java\jdk1.8.0_102\bin\javadoc.exe*).

2. If we want to show the documentation for all private and public fieilds and methods, then we have to select "Private" on the shown dialog box.
3. Make sure to select the destination folder where the generated documentation files will be stored. Theh default would be a folder called 'doc' under your project folder in eclipse workspace. Click 'Next.'
4. The next wizard window will allow you to customize the contents of the generated documentation:
5. Click the 'Finish' button and watch the progress in your console view.
6. Select your project within the package explorer view and refresh it by pressing F5; you should see the doc folder.
7. Double click on the 'index.html' fle to open the main page for the generated documentation. You may also browse your documentation by directly opening the index.html filie using the Internet Explorer.



Additional Practice Problems

Exercise 7: Working with Prime Numbers

Write an application that asks the user to enter an integer number and checks if that number is *prime* or not.

- Modify your application to print all prime numbers between 1 and 100.
- Modify your application to count the number of prime numbers between 1 and 1000.

Exercise 8: Hypotenuse Calculations

Define a method hypotenuse that calculates the hypotenuse of a right triangle when the lengths of the other two sides are given.

$$c = \sqrt{(a^2 + b^2)}$$

The method should take two arguments of type double and return the hypotenuse as a double. Incorporate this method into an application that reads values for side1 and side2 and performs the calculation with the hypotenuse method.

Use Math methods pow and sqrt to determine the length of the hypotenuse for each of the triangles in Table. [Note: Class Math also provides method hypot to perform this calculation.]

Triangle	Side 1	Side 2
1	3.0	4.0
2	5.0	12.0
3	8.0	15.0

Exercise 9: The Rectangle

Create a class named **'Rectangle'** with two data members- **length** and **width** and a method to calculate the area which is **'length*width'**. The class has three constructors, which are:

1 - **Having no parameter** - values of both length and width are assigned zero.

2 - **Having two numbers as parameters** - the two numbers are assigned as length and width respectively.

3 - **Having one number as parameter** - both length and width are assigned that number.

Now, create a **RectangleTestClass** and Instantiate objects of the 'Rectangle' class having none, one and two parameters and print their areas.